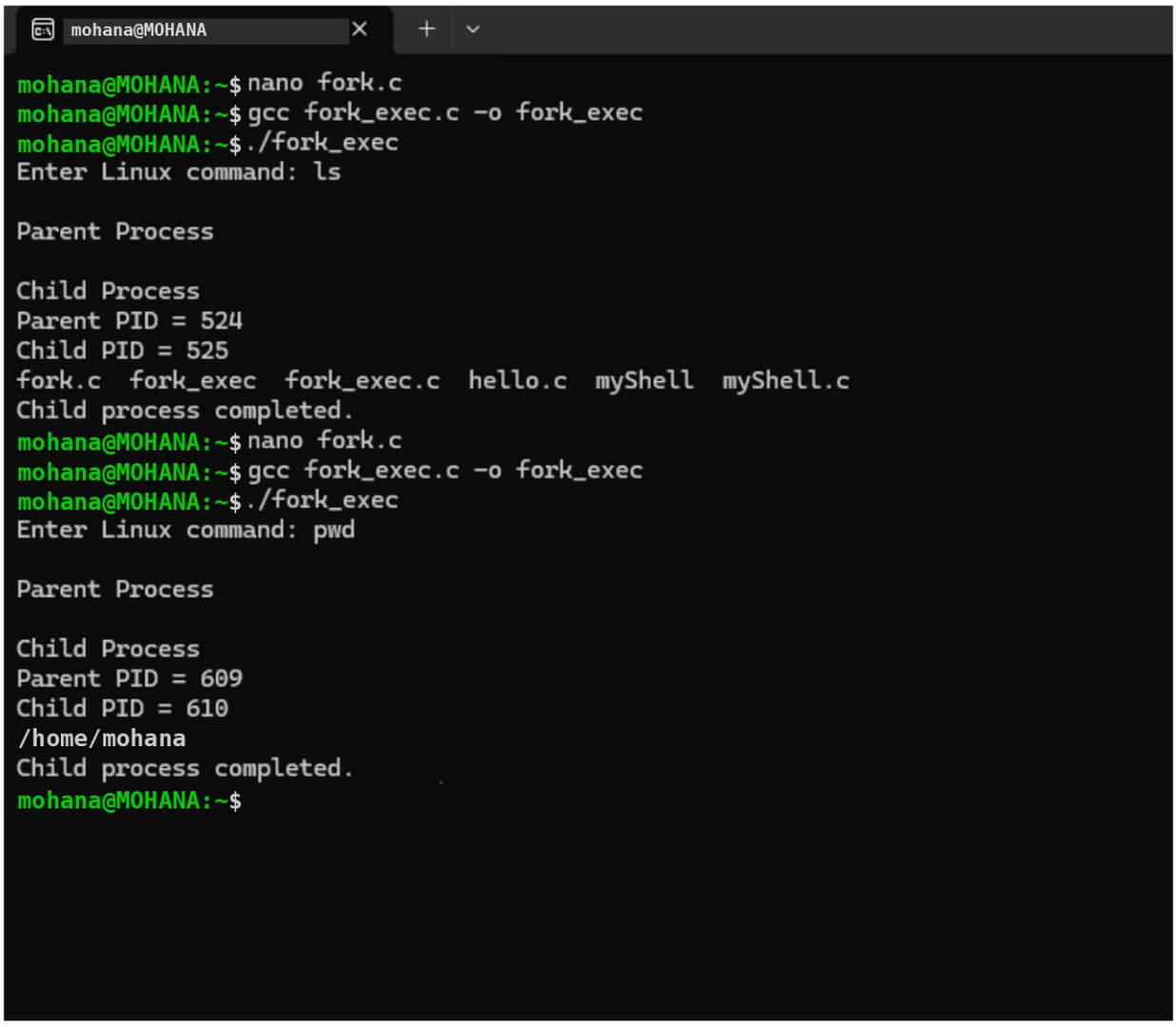
GNU nano 7.2 fork.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    char command[100];

    // Accept Linux command from user
    printf("Enter a Linux command: ");
    scanf("%s", command);

    // Create a child process
    pid = fork();

    if (pid < 0)
    {
        // Fork failed
        printf("Fork failed!\n");
        exit(1);
    }
    else if (pid == 0)
    {
        // Child Process
        printf("\n===== Child Process =====\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        // Execute the entered command
        execlp(command, command, (char *)NULL);

        // This executes only if exec fails
        printf("Exec failed!\n");
        exit(1);
    }
    else
    {
        // Parent Process
        printf("\n===== Parent Process =====\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        // Wait for child process to finish
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
```


mohana@MOHANA
mohana@MOHANA:~$ nano fork.c
mohana@MOHANA:~$ gcc fork_exec.c -o fork_exec
mohana@MOHANA:~$ ./fork_exec
Enter Linux command: ls
Parent Process
Child Process
Parent PID = 524
Child PID = 525
fork.c fork_exec fork_exec.c hello.c myShell myShell.c
Child process completed.
mohana@MOHANA:~$ nano fork.c
mohana@MOHANA:~$ gcc fork_exec.c -o fork_exec
mohana@MOHANA:~$ ./fork_exec
Enter Linux command: pwd
Parent Process
Child Process
Parent PID = 609
Child PID = 610
/home/mohana
Child process completed.
mohana@MOHANA:~$

GNU nano 7.2 fork2.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    pid_t pid;

    while (1)
    {
        printf("\nEnter Linux Command (or type exit to quit): ");
        fgets(command, sizeof(command), stdin);

        // Remove newline character
        command[strcspn(command, "\n")] = '\0';

        // Exit condition
        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting Program...\n");
            break;
        }

        pid = fork();

        if (pid < 0)
        {
            printf("Fork Failed!\n");
            exit(1);
        }
        else if (pid == 0)
        {
            // Child Process
            printf("\nChild Process Created\n");
            printf("Child PID  : %d\n", getpid());
            printf("Parent PID : %d\n", getppid());

            execlp(command, command, (char *)NULL);

            // Executes only if exec fails
            printf("Invalid Command!\n");
            exit(1);
        }
        else
        {
            // Parent Process
            wait(NULL);
            printf("Command Execution Completed.\n");
        }
    }

    return 0;
}
```

```
mohana@MOHANA:~$ gcc fork2.c -o fork2
mohana@MOHANA:~$ ./fork2

Enter Linux Command (or type exit to quit): date

Child Process Created
Child PID  : 757
Parent PID : 756
Invalid Command!
Command Execution Completed.

Enter Linux Command (or type exit to quit): pwd

Child Process Created
Child PID  : 758
Parent PID : 756
/home/mohana
Command Execution Completed.

Enter Linux Command (or type exit to quit): exit
Exiting Program...
mohana@MOHANA:~$
```